

Guide complet de la bibliothèque yfinance en Python

1. Introduction

`yfinance` est une bibliothèque Python permettant de récupérer facilement des données financières depuis Yahoo Finance. Elle est très utilisée en **finance quantitative**, **data science**, **analyse de marché**, et pour des projets académiques (pricing, backtesting, risk management).

Ce document présente **de manière exhaustive** les principales commandes et fonctionnalités de `yfinance`, avec des exemples clairs et directement exploitables en Python.

2. Installation et import

Installation

```
pip install yfinance
```

Import standard

```
import yfinance as yf
```

3. Objet central : `Ticker`

La classe principale de `yfinance` est `Ticker`, qui représente un actif financier (action, indice, ETF, crypto, etc.).

Création d'un ticker

```
aapl = yf.Ticker("AAPL")
```

4. Téléchargement des données de prix

4.1 Méthode `history()`

Permet de récupérer l'historique des prix.

```
data = aapl.history(period="1y")
```

Paramètres principaux

Paramètre	Description
<code>period</code>	Période globale (<code>1d</code> , <code>5d</code> , <code>1mo</code> , <code>3mo</code> , <code>6mo</code> , <code>1y</code> , <code>5y</code> , <code>max</code>)
<code>start</code>	Date de début (<code>YYYY-MM-DD</code>)
<code>end</code>	Date de fin
<code>interval</code>	Fréquence (<code>1m</code> , <code>5m</code> , <code>1h</code> , <code>1d</code> , <code>1wk</code> , <code>1mo</code>)
<code>auto_adjust</code>	Ajuste automatiquement les prix (dividendes/splits)
<code>actions</code>	Inclure dividendes et splits

Exemple avancé :

```
data = aapl.history(start="2022-01-01", end="2023-01-01", interval="1d")
```

4.2 Fonction globale `yf.download()`

Utile pour télécharger **plusieurs actifs à la fois**.

```
data = yf.download(["AAPL", "MSFT", "GOOG"], period="2y")
```

Options supplémentaires :

```
data = yf.download("AAPL", period="1y", interval="1d", group_by="ticker")
```

5. Données fondamentales

5.1 Informations générales

```
aapl.info
```

Contient : - Nom de l'entreprise - Secteur - Capitalisation boursière - PER - Rendement du dividende - Description

 Attention : certaines clés peuvent être manquantes.

5.2 États financiers

Compte de résultat

aapl.financials

Compte de résultat trimestriel

aapl.quarterly_financials

Bilan

aapl.balance_sheet

Bilan trimestriel

aapl.quarterly_balance_sheet

Flux de trésorerie

aapl.cashflow

6. Dividendes et splits

Dividendes

aapl.dividends

Splits

aapl.splits

Actions sur titres (dividendes + splits)

aapl.actions

7. Données de marché avancées

7.1 Volume et prix en temps quasi réel

```
aapl.fast_info
```

Inclut : - last_price - market_cap - open - previous_close

7.2 Recommandations des analystes

```
aapl.recommendations
```

7.3 Calendrier (earnings, dividendes)

```
aapl.calendar
```

8. Données options (Options Chain)

Récupération des maturités

```
aapl.options
```

Chaîne d'options complète

```
opt = aapl.option_chain("2024-06-21")  
calls = opt.calls  
puts = opt.puts
```

Colonnes importantes : - strike - lastPrice - bid / ask - impliedVolatility - openInterest

9. Indices, ETF et cryptomonnaies

Indices

```
sp500 = yf.Ticker("^GSPC")
```

ETF

```
spy = yf.Ticker("SPY")
```

Cryptomonnaies

```
btc = yf.Ticker("BTC-USD")
```

10. Gestion des données avec Pandas

Les objets retournés sont des **DataFrames Pandas**.

Exemple :

```
returns = data['Close'].pct_change()
```

Calcul de volatilité :

```
vol = returns.std() * np.sqrt(252)
```

11. Bonnes pratiques

- Toujours vérifier la disponibilité des données
- Ne pas dépendre de Yahoo Finance pour des applications critiques
- Mettre en cache les données
- Vérifier les unités (USD, millions, milliards)

12. Limites de yfinance

- Pas de garantie de fiabilité
- Données parfois incomplètes
- API non officielle
- Pas adaptée au trading haute fréquence

13. Conclusion

`yfinance` est une bibliothèque **extrêmement puissante pour l'analyse financière**, idéale pour : - Projets étudiants - Recherche académique - Prototypage quantitatif

Elle constitue une excellente base avant de passer à des API professionnelles (Bloomberg, Refinitiv, Quandl).

Document rédigé pour Overleaf – compatible LaTeX Markdown